

ANALYSIS OF AN EFFICIENT CIM-SRAM FOR VLSI APPLICATIONS

Mohamed Fawaz. D
Department of Electronics and
communication
Sathyabama Institute of
Science and Technology
Chennai, India
dfawaz2003@gmail.com

Dhanush. M
Department of Electronics and
communication
Sathyabama Institute of
Science and Technology
Chennai, India
dhanushmohanarangan@gmail.com

Dr. G. Sowmiya
Department of Electronics and
communication
Sathyabama Institute of
Science and Technology
Chennai, India
sowmiya.g.ece@sathyabama.ac.in

B. Aswin
Department of Electronics and
Communication
Sathyabama Institute of
Science and Technology
Chennai, India
aswinbalaji1830@gmail.com

Rishikesh.SS
Department of Mechatronics
Engineering
Hindustan Institute of Technology
and Science
Chennai, India
rishikeshss853@gmail.com

Dashwanth.P
Department of Electronics and
communication
Sathyabama Institute of
Science and Technology
Chennai, India
dashwantharun008@gmail.com

Abstract- The traditional SRAM suffers from high data movement between memory and processing units which leads to increase in power consumption and latency in data-intensive workloads. COMPUTE-IN-MEMORY(CIM) addresses this limitation by allowing computation within the memory array. Therefore, this work presents an analog CIM-SRAM architecture implemented in CMOS VLSI using a standard 6T SRAM cell. A 4*4 CIM-SRAM array is designed to perform in-memory MAC operations by exploiting analog bit-line accumulation for parallel computation without modifying the basic cell. Simulations are done to evaluate power consumption, delay and power delay product (PDP). When comparing with conventional SRAM operation, this proposed design can achieve lower power consumption and reduced PDP while maintaining reliable memory functionality. These observations indicate that the proposed CIM-SRAM architecture is suitable for low-power, high-throughput edge AI systems.

Keywords- CIM-SRAM, 6T SRAM, Low-power, Delay, PDP.

I. INTRODUCTION

The high rate of artificial intelligence and machine learning development, as well as data-centric applications development, have led to a high-performance and energy-efficient computing system demand. Traditional SRAM memory architectures separate memory and processing unit and therefore, there is a need to transfer data between the memory and processing unit frequently. This constant traffic of data leads to the further consumption of power, growing latency and the lack of throughput in the system which is also known as the memory

wall problem [1].

A modern standard processor uses Static Random-Access Memory (SRAM) in caches and on-chip memories because it is fast and can be built using standard CMOS fabrication methods [1]. Nonetheless, the traditional SRAM is only meant to store data rather than to compute. Repeated read and write operations further add to the consumption of energy and poor efficiency of the system especially when large data sets are involved.

Compute-in-Memory (CIM) architectures are meant to address these issues by placing computation units in the memory array. CIM is much more efficient because it can carry out operations like multiplication and accumulation within memory thus

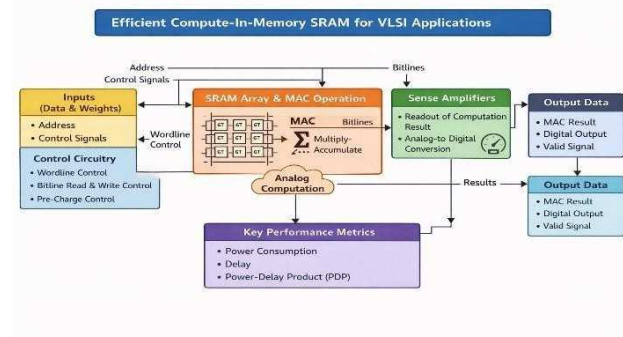


Fig 1. CIM-SRAM BLOCK DIAGRAM

ensuring that the movements of data are limited and it is also highly efficient in terms of power consumption [1][5]. CIM architectures based on SRAM are of particular interest since they enable the integration of computation in any existing memory data structure and do not need dramatic modifications to the process.

There are broadly digital CIM and analog CIM implementations of CIM. Digital CIM has high accuracy but is associated with large circuit complexity and area overhead. Analog CIM instead takes advantage of the inherent analog characteristics of memory bit lines to do parallel computations at much lower power consumption. This causes analog CIM-SRAM to be an ideal fit to applications where limited precision can be accepted, e.g. neural networks and edge intelligence systems [2][3][6][7][8].

An analog CIM-SRAM design of a traditional 6T SRAM cell is suggested and discussed in this paper. The design allows in-memory computation and has a reliable read and write operation. With CMOS VLSI technology, a small-scale CIM-SRAM array is designed (**Fig 10**) and its functionality is tested by simulation. This work aims at showing a CIM-SRAM design that is energy efficient, stable and has a high computational capability and low power consumption.

II. RELATED WORK

Several SRAM based CIM architectures have been reported to discuss the memory wall problem in data-storage applications. Chen et al. Introduced a charge-domain 6T SRAM CIM macro enabling and programmable CNN interference with improved energy efficiency[1].Sinangil et al. demonstrated a 7-nm CIM-SRAM macro supporting multi-bit inputs and weights for achieving high throughput for edge AI systems[2].Su et al.Presented a 28-nm 6T SRAM CIM macro with 8-bit precision, showing that the standard SRAM arrays can be opted for in-memory MAC operations without doing any significant cell modification[3].Other works discussed 8T/9T SRAM cells and charge-sharing or time-domain techniques for improving computational linearity and robustness at the expense of increased area overhead[5][8].These research prove that the SRAM-based CIM is a promising option for low-power and high-throughput VLSI systems.

III. PROPOSED CIM-SRAM ARCHITECTURE

The suggested Compute-in-Memory architecture is created on top of an existing 6T SRAM cell, which is system-level adjusted to support in-memory computation without having to change the basic cell. The key task of such design is to implement multiply-and-accumulate (MAC) instruction in the SRAM array itself and still ensure the reliability of the read and write performance [2][3][6].

To prove the viability of the offered strategy, a small scale CIM-SRAM array of 4 on 4 is used. That is, a binary weight is stored in each SRAM cell, and computations are made by taking advantage of the analog response of the bit line when reading. The complete architecture comprises of SRAM array, control circuits of wordline, bit line, pre-charge, write drivers, and sensing [1][6][8].

During normal memory operation, the SRAM cells function as standard storage elements. During computation mode, the same cells are reused to perform analog operations, thereby eliminating the need for separate processing units and reducing data movement overhead.

A. Conventional 6T SRAM Cell Operation

The 6T SRAM cell comprises of a bistable latch made of two cross-coupled CMOS inverters and two transistors on the access chroma under control by the wordline. The data stored remains there as long as power is present thus its non-volatile behavior is experienced during the active operation [13].

The write operations are complemented by overwriting the bit lines with the complementary data and the wordline is enabled. The operation of read, includes pre-charging of the bit lines and measuring the voltage difference as a result of the stored data when the wordline is activated. Performance of the cell in the read and write action is a design issue of high concern and is determined using the PDP [1].

The proposed design takes advantage of the behavior of an intrinsic read of the 6T SRAM cell to aid the computation process by enabling the execution of arithmetic operations without necessarily changing the number of transistors in the cell.

B. Compute-in-Memory Operation

In compute mode the input data is during the form of analog level of input voltage in the form of bit lines, and the wordline of the row of interest is enabled. Each SRAM cell stores a weight of binary data, which either permits or bars current flow through the access transistor, depending on the weight stored in the cell. Which in turn is the product of the input voltage and the weight stored in the cell [2][6][8]. Since several cells in a column can be triggered at once, the current in them is automatically added to the bit line, which has the effect of a multiply-and-accumulate operation [1][3][6]. This built in current summation feature makes it possible to bend the computation in parallel inside the memory array. The summation of analog signal on the bit line can then be converted into a digital output with the help of a sensing circuit. The method greatly lowers the accesses to memory needed to compute and energy efficiency in general.

C. Peripheral Circuits

In order to have a stable operation of CIM, there are a few peripheral circuits incorporated into the design:

- **Pre-charge Circuit:** Pre-charges the bit lines to a defined voltage level before read or compute operations to ensure consistent sensing.
- **Write Driver:** Drives appropriate voltage levels onto the bit lines during write operations to store weight data accurately.
- **Worldline Driver:** Selects the appropriate row during read, write, or compute operation
- **Sense Amplifier:** Detects the voltage or current variation on the bit line and converts the analog computation result into a digital output.

Such peripheral circuits are also critically crafted to consume minimum extra power and still ensure the proper functionality under memory mode and compute mode [3][6][10].

IV. SIMULATION RESULTS AND DISCUSSION

The specified CIM-SRAM architecture is tested on the circuit-level simulations that take place in the CMOS VLSI design tool. Each and every simulation is carried out to ensure that there is a proper functionality of memory, compute-in-memory, and also performance in the way of power, delay and stability.

A. Delay Margin Analysis

The delay is used to analyze the speed and performance of the SRAM cell when it is being read. The delay can be obtained by plotting of the butterfly curve of the voltage transfer characteristics of the present cross-coupled inverters of the cell (**Fig 2,3**). The results of the simulation showed that the delay through the cell (**Fig 2,3**) was 40.52ns, which confirms the dependable performance and speed. The obtained delay value demonstrates that the analog compute operation does not significantly degrade the stability of the memory cell when compared to conventional SRAM (**Fig 4,5,6**) operation. The improved delay ensures that the proposed CIM-SRAM design can operate safely under process and voltage variations, making it suitable for low-power VLSI applications.

B. Transient and Compute Operation Analysis

Transient simulations are performed to validate the compute-in-memory functionality of the proposed design. During computation, analog input voltages are applied to the bit lines, and the corresponding word lines are activated to enable current flow based on the stored weight values. The resulting bit line current represents the multiplication of the input voltage and the stored data. When multiple cells in a column are activated, the individual currents are accumulated on the bit line, effectively realizing an in-memory multiply- and- accumulate operation. The transient waveforms confirm correct accumulation behavior and demonstrate

the feasibility of performing parallel computation within the SRAM array without additional processing circuit (**Fig 7,8,9**).

C. Power and Performance Evaluation

Transient simulations of both cell-based and full CIM-SRAM array (array) are conducted to obtain power and delay values. The power-delay product (PDP) is a quantification of what is used to determine the performance of the performance, giving a complete measure of the energy efficiency. The 4x4 CIM-SRAM array proposed has a PDP of 0.208pj and a single SRAM cell has the PDP of 1.233pj. These demonstrations show that there is a huge saving of energy usage when compared with the traditional SRAM system of doing the computing [2][6][8][9]. According to the simulation data, the proposed design has a computational power of 10.71mW, which is why it is appropriate to use it in products with energy-limiting requirements like edge control and embedded artificial intelligence applications (**Table 1**).

V. RESULTS

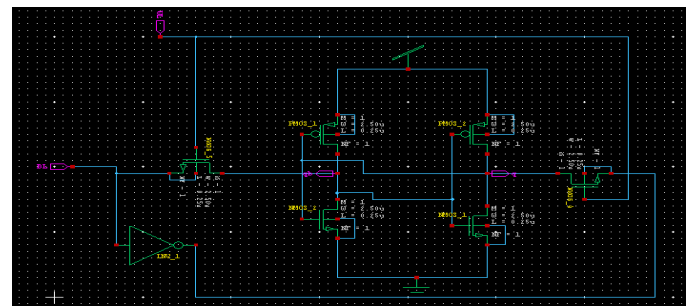


Fig 2. SRAM 6T CELL

This figure shows the conventional 6T SRAM which consists of two cross coupled inverters and two access transistors which forms the basic storage element used for the operations in proposed architecture.

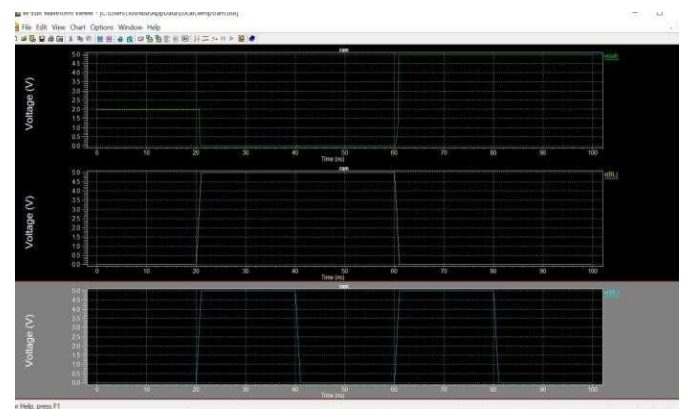


Fig 3. SRAM 6T WAVEFORM

This waveform shows the read and write behavior of 6T SRAM cell and confirms stable data storage and proper switching during memory operations.

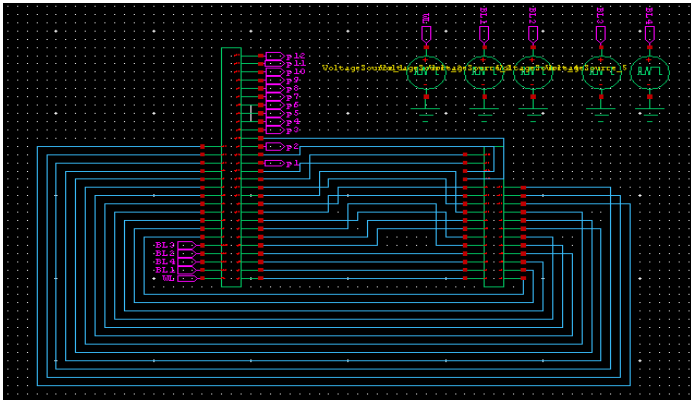


Fig 4. TRADITIONAL SRAM MEMORY

This figure shows the block level structure of a traditional SRAM without CIM capability and also shows the separation between memory and processing units.

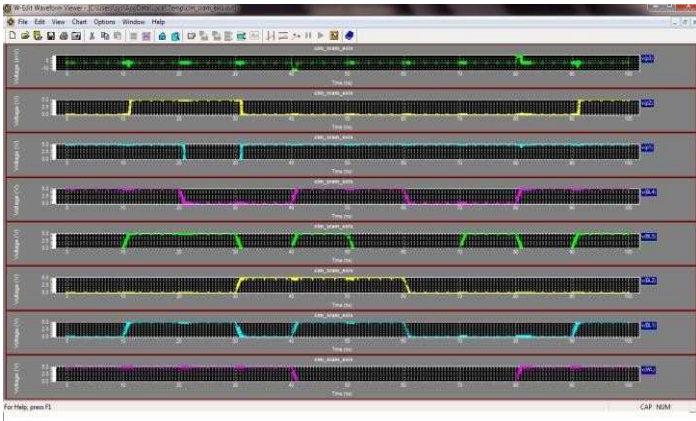


Fig 5. TRADITIONAL SRAM WAVEFORM

This waveform shows the behavior of the traditional SRAM during read/write operations. It shows higher delay and power consumption compared to CIM.

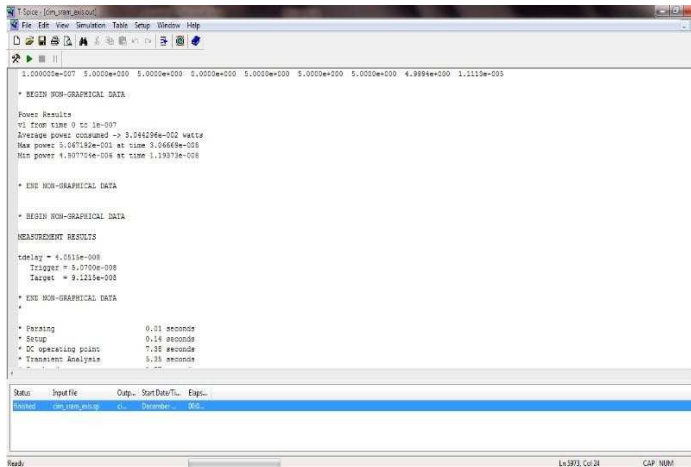


Fig 6. TRADITIONAL SRAM RESULT

This result summarizes the performance of traditional SRAM as it demonstrates higher power and PDP due frequent data movement.

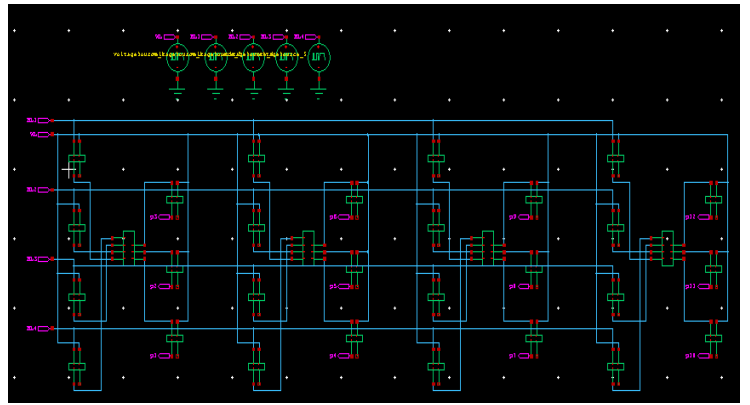


Fig 7. PROPOSED CIM SRAM DESIGN

This figure presents the architecture of the proposed CIM-SRAM design which allows in memory computation without modifying the SRAM cell.

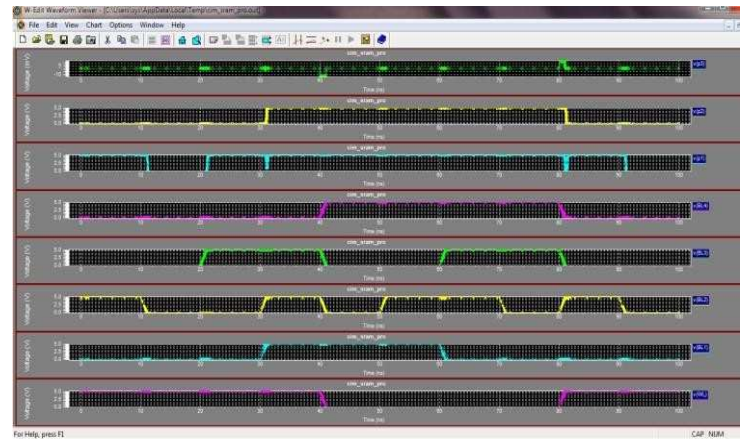


Fig 8. PROPOSED CIM SRAM WAVEFORM

This waveform shows the CIM operation where analog inputs are applied to bit lines and validating MAC operation successfully inside the memory array.

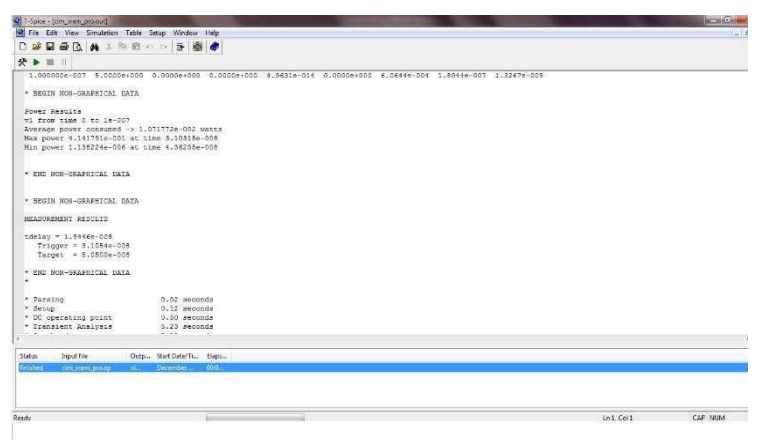


Fig 9. PROPOSED CIM SRAM RESULT

This result summarizes the simulation results of CIM-SRAM array. It shows reduced power consumption and improved delay when compared to traditional SRAM.

COMPARISION TABLE

Parameters	Power(mW)	Delay(ns)	PDP(PJ)
Conventional SRAM	30.44	40.52	1.233
CIM-SRAM	10.71	19.44	0.208

Table 1

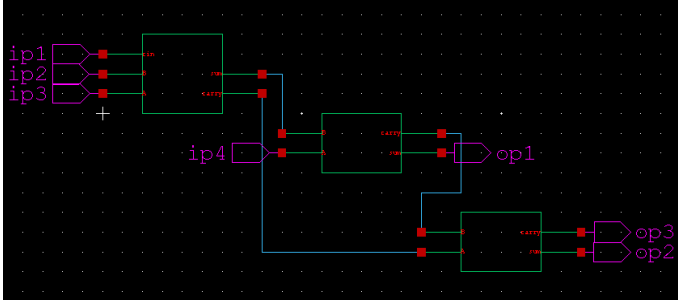


Fig 10. COMPUTATIONAL UNIT

This unit shows how multiple SRAM cells perform parallel MAC operations collectively and highlights the main advantage of CIM in achieving high throughput and energy efficiency.

VI. LIMITATIONS

Although the proposed CIM-SRAM design gives improved energy efficiency and reduced power delay product, there are certain limitations which should be acknowledged. First the design heavily relies on analog computation which is inherently sensitive to process, voltage and temperature (PVT) variations which can affect computational accuracy. Second, the proposed implementation is validated only on a small scale 4*4 array so implementing it into larger arrays can give challenges like increased noise and sensing inaccuracies [11].

VII. CONCLUSION

This paper has also introduced an efficient architectural design of a Compute-in-Memory SRAM architecture on a typical 6T SRAM cell and discussed it through CMOS VLSI technology. The proposed design allows in-memory computation, by taking advantage of the analog behavior of the SRAM bit lines, and thus decreases the overhead of a data movement incurred by a traditional von Neumann architecture. A 4×4 CIM-SRAM array was developed to do multiplies and accumulation right in the memory array. Circuit level simulation was done to test stability, power consumption and performance. The CIM SRAM cell as suggested had a delay of 19.44ns thus indicating that the cell can work reliably both when accessing the memory and when computing. In addition, it has a power-delay product of 0.208pj and a power efficiency of 10.71 mW, which is among the reasons to conclude that the proposed CIM-SRAM architecture fits the purpose of low- power computing with moderate precision and high throughput. The expansion of the array size and the incorporation of the design in the larger neural network accelerators can be considered in future work to further tests.

VIII. FUTURE WORK

Future research will focus on taking the proposed CIM-SRAM architecture onto larger memory arrays to validate system level performance and efficiency. This design will help in applications for complex network models. This work will also have layout level implementation and post layout simulations to evaluate area, power and timing under realistic conditions. Finally, embedding the proposed CIM-SRAM within a full network framework and benchmarking it against traditional SRAM architecture will provide deeper insights into its real-world performance for edge AI applications [12].

IX. SUMMARY

This work presents an efficient CIM-SRAM design based on a conventional 6T SRAM cell using CMOS VLSI technology. The proposed design enables in-memory operations with reduced data movement overhead. Simulation results show improved power efficiency, reduced delay and a low power delay product when compared to traditional SRAM operation. These results suggest that the proposed CIM-SRAM architecture is well suited for low power, edge AI and data-based computing applications.

ACKNOWLEDGEMNT

We would like to sincerely thank Dr. G. Sowmiya, our project guide, for her essential advice and unwavering support during this effort. We also acknowledge the management of SATHYABAMA INSTITUTE OF SCIENCE AND TECHNOLOGY and the Department of Electronics and Communication Engineering for providing the required facilities and assistance.

REFERENCES

- [1] Z. Chen et al., "CAP-RAM: A Charge-Domain In-Memory Computing 6T-SRAM for Accurate and Precision-Programmable CNN Inference," *IEEE Journal of Solid-State Circuits (JSSC)*, 2021. DOI: 10.1109/JSSC.2021.3056447.
- [2] M. E. Sinangil et al., "A 7-nm Compute-in-Memory SRAM Macro Supporting Multi-Bit Input, Weight and Output and Achieving 351 TOPS/W and 372.4 GOPS," *IEEE Journal of Solid-State Circuits (JSSC)*, 2021.
- [3] J.-W. Su et al., "A 28nm 384kb 6T-SRAM Computation-in-Memory Macro with 8b Precision for AI Edge Chips," *IEEE International Solid-State Circuits Conference (ISSCC)*, 2021.
- [4] S. Huang et al., "Secure XOR-CIM Engine: Compute-In Memory SRAM Architecture with Embedded XOR Encryption," *IEEE Transactions on VLSI Systems (TVLSI)*, vol. 29, no. 12, pp. 2027–2039, 2021.

- [5] K. Lee et al., "A Charge-Sharing Based 8T SRAM In-Memory Computing for Edge DNN Acceleration," *ACM/IEEE Design Automation Conference (DAC)*, 2021. DOI: 10.1109/DAC18074.2021.9586103.
- [6] C. Yu et al., "A 65-nm 8T SRAM Compute-in-Memory Macro with Column ADCs for Processing Neural Networks," *IEEE Journal of Solid-State Circuits (JSSC)*, 2022. DOI: 10.1109/JSSC.2022.3162602.
- [7] X. Zhang et al., "A Local Transpose 9T SRAM Compute-In-Memory Macro with Programmable Single-Slope SAR ADC," *2022 IEEE Asian Solid-State Circuits Conference (A-SSCC)*, 2022. DOI: 10.1109/A-SSCC56115.2022.9980672.
- [8] H. Wang et al., "A Charge Domain SRAM Compute-in-Memory Macro With C-2C Ladder-Based 8-Bit MAC Unit in 22-nm FinFET Process for Edge Inference," *IEEE Journal of Solid-State Circuits (JSSC)*, 2023. DOI: 10.1109/JSSC.2022.3232601.
- [9] V. Damodaran et al., "SRAM In-Memory Computing Macro with Delta-Sigma Modulator-Based Variable-Resolution Activation," *IEEE Solid-State Circuits Letters (SSCL)*, vol. 6, pp. 293–296, 2023.
- [10] H. Kim et al., "A 1–16b Reconfigurable 80Kb 7T SRAM-Based Digital Near-Memory Computing Macro for Processing Neural Networks," *IEEE Transactions on Circuits and Systems I (TCAS-I)*, vol. 70, no. 4, pp. 1580–1590, 2023. DOI: 10.1109/TCSI.2022.3232648.
- [11] H. Jeong et al., "A Ternary Neural Network Computing-in-Memory Processor With 16T1C Bitcell Architecture," *IEEE Transactions on Circuits and Systems II (TCAS-II): Express Briefs*, vol. 70, no. 5, pp. 1739–1743, May 2023.
- [12] C. Xue et al., "A 768.7–2124.2 TOPS/W Time-Domain Computing-in-Memory Macro with Low Static Leakage and Precision-Configurable TDC," *IEEE TCAS-II: Express Briefs*, 2024.
- [13] J.-W. Su et al., "8-Bit Precision 6T SRAM Compute-in-Memory Macro Using Global Bitline...", *IEEE Transactions on Circuits and Systems I (TCAS-I)*, vol. 71, no. 4, 2024 (paper page shows venue/volume/issue).